Computational Thinking

Week-9 Summary

Introduction to Graph data structure(Lecture 9.1):

- Relationship between two student in subject marks S

```
          S
A ----------------> B          Difference is between [10, 20]

          M
11 ----------------> [7, 16, 6...]
          M
 1  ----------------> [3, 12, 24...]
```

- Same is applicable for Physics and Chemistry

- Seq.No. 11 having paths/edges to 6, 7 and 16

Conclusion:
Define graph data structure.
Construct graph data structure from the list.
Define node or vertex and edge.
Assign directions and labels to the edges.
Advantages of drawing graph data structure.

Introduction to matrices and implementation of matrix using nested dictionary(Lecture 9.2):

We need a matrix:
It's two dimensional table m rows, n columns. Random access to matrix[i][j]. By convention, rows and columns are numbered from 0

$0 \le i \le m - 1, 0 \le j \le n - 1$

Create matrix procedure:

```
Procedure CreateMatrix(rows,cols)
        mat = {}
        i = 0
        while (i < rows) {
                mat[i] = {}
                j = 0
                while (j < cols){
                        mat[i][j] = 0
                        j = j + 1
                }
                i = i + 1
        }
        return(mat)
```

```
End CreateMatrix
```

Iterating through rows and columns:

```
foreach r in sort(keys(mymatrix)) {
        foreach c in sort(keys(mymatrix[0])) {
                Do something with mymatrix[r][c]
        }
}
```

To improve readability, use rows() and columns()

```
foreach r in rows(mymatrix) {
        foreach c in columns(mymatrix) {
        Do something with mymatrix[r][c]
}
}
```

Undirected graphs and cliques(Lecture 9.3):

- Mutual help between A to B and B to A.
- A could mentor B in Maths
- B could mentor A in Physics

Clique: All connected students we call them as group/clique. Let's take undirected edges A - B - C - A, it forms a clique/group. They exclude others from their group.

Concept of popular students using Graph(Lecture 9.4):

- Earlier graph build in 9.3 can be used to identify popular students. A student is popular if he/she can help others or get help from others. In this way this student is popular.

- How many incoming edges are there is called degree(in/out)

- Take a pool/set of cards then find mentor and mentee relationship between students with a difference of 10 to 20 only. Then we can find in/out edges called degree

---

Pseudocode for realtime examples using graphs(Lecture 9.5):

Create a dictionary for marks in subject:

```
mathMarks = ReadMarks(Mathematics)


Procedure ReadMarks(subj)
        marks = {}
```

```
        while (Table 1 has more rows) {
            Read the first row X in Table 1
            marks[X.Seqno] = X.subj
        }
        Move X to Table 2
        return(marks)
    End ReadMarks
```

Create a mentoring graph for a subject:

Represent as a matrix

M[i][j] = 1 — edge from i to j

M[i][j] = 0 — no edge from i to j

```
    Procedure CreateMentorGraph(marks)
        n = length(keys(marks))
        mentorGraph = CreateMatrix(n,n)
        foreach i in keys(marks){
            foreach j in keys(marks){
                ijMarksDiff = marks[i] – marks[j]
                if (10 ⩽ ijMarksDiff and ijMarksDiff ⩽ 20) {
                    mentorGraph[i][j] = 1
                }
            }
        }
        return(mentorGraph)
    End CreateMentorGraph(marks)
```

Use mentor graphs to pair off students:

```
    mathMarks = ReadMarks(Mathematics)
    phyMarks = ReadMarks(Physics)
    mathMentorGraph = CreateMentorGraph(mathMarks)
    phyMentorGraph = CreateMentorGraph(phyMarks)
    paired = {}
    foreach i in rows(mathMentorGraph) {
        foreach j in columns(mathMentorGraph) {
```

```
            if (mathMentorGraph[i][j] == 1 and phyMentorGraph[j][i] == 1 and
        not(isKey(paired,i)) and not(isKey(paired,j))) {
                    paired[i] = j
                    paired[j] = i
                    }
            }
        }
```

– A student who can be mentored by many other students is popular

– Create mentoring graphs for all three subjects

```
        mathMarks = ReadMarks(Mathematics)
        phyMarks = ReadMarks(Physics)
        chemMarks = ReadMarks(Chemistry)
        mathMentorGraph = CreateMentorGraph(mathMarks)
        phyMentorGraph = CreateMentorGraph(phyMarks)
        chemMentorGraph = CreateMentorGraph(chemMarks)
```

- Count incoming edges for each student

```
        popularity = {}
        foreach i in keys(mathMarks) {
                popularity[i] = 0
                foreach j in keys(mathMarks) {
                        if (mathMentorGraph[j][i] == 1){
                                popularity[i] = popularity[i] + 1
                        }
                        if (phyMentorGraph[j][i] == 1){
                                popularity[i] = popularity[i] + 1
                        }
                        if (chemMentorGraph[j][i] == 1){
                                popularity[i] = popularity[i] + 1
                        }
                }
        }
```